

Exercise 01:

Write a program that reads a Celsius degree and converts it to Fahrenheit.

Formula: $F = (9/5) * Celsius + 32$.

```
import java.util.Scanner;
public class Exercise1 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter a degree in Celsius: ");
        double celsius = input.nextDouble();
        double fahrenheit = (9.0 / 5) * celsius + 32;
        System.out.println(celsius + " in Celsius is " + fahrenheit + " Fahrenheit");
    }
}
```

Output

```
Enter a degree in Celsius: 43
43.0 in Celsius is 109.4 Fahrenheit
```

Exercise 02:

Write a program that asks the user to enter the starting velocity v0, the ending velocity v1 and the time span t, and

then displays the average acceleration with two digits after the decimal point using the formula: $a = (v1 - v0)/t$

```
import java.util.Scanner;
public class Exercise2 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter v0: ");
        double v0 = input.nextDouble();
        System.out.print("Enter v1: ");
        double v1 = input.nextDouble();
        System.out.print("Enter t: ");
        double t = input.nextDouble();
        double a = (v1 - v0) / t;
        System.out.printf("The average acceleration is %.2f", a);
    }
}
```

Output

```
Enter v0: 5.5
Enter v1: 50.9
Enter t: 4.5
The average acceleration is 10.09
```

Exercise 03:

Write a program that displays the following on the screen:

```
1 1
2 4
3 9
4 16
5 25
```

You should use Math.pow method.

```
public class Exercise3 {
    public static void main(String[] args) {
        for (int i = 1; i <= 5; i++) {
            System.out.println(i + " " + (int)Math.pow(i, 2));
        }
    }
}
```

Output
1 1
2 4
3 9
4 16
5 25

Exercise 04:

Write a program that asks the user to enter a radius of a circle and then displays its area and its perimeter.

Area: $\text{PI} \times r^2$

Area: $2 \times \text{PI} \times r$

```
import java.util.Scanner;
public class Exercise4 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter the radius: ");
        double radius = input.nextDouble();
        double area = Math.PI * Math.pow(radius, 2);
        double perimeter = 2 * Math.PI * radius;
        System.out.println("Area = " + area);
        System.out.println("Perimeter = " + perimeter);
    }
}
```

Output
Enter the radius: 5
Area = 78.53981633974483
Perimeter = 31.41592653589793

Exercise 05:

Write a program that asks the user to enter character and then displays its ASCII code.

```
import java.util.Scanner;
public class Exercise5 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter a character: ");
        char ch = input.next().charAt(0);
        System.out.println("ASCII code = " + (int) ch);
    }
}
```

Output
Enter a character: A
ASCII code = 65

Exercise 06:

Given the following:

- One birth every 7 seconds,
- One death every 13 seconds,
- One new immigrant every 45 seconds,
- Current population:1000000
- One year: 365 days

Write a program that displays the population for each of the next five years.

```
public class Exercise6 {
    public static void main(String[] args) {
        int population = 1000000;
        int birthsPerYear = (365 * 24 * 60 * 60) / 7;
        int deathsPerYear = (365 * 24 * 60 * 60) / 13;
        int immigrantsPerYear = (365 * 24 * 60 * 60) / 45;
        for (int year = 1; year <= 5; year++) {
            population = population + birthsPerYear - deathsPerYear + immigrantsPerYear;
            System.out.println("Year " + year + ": " + population);
        }
    }
}
```

Output
Year 1: 2861285
Year 2: 4722570
Year 3: 6583855
Year 4: 8445140
Year 5: 10306425

Exercise 07:

Write a program that asks the user to enter an integer and then displays if it is even or odd. You should use switch case.

```
import java.util.Scanner;
public class Exercise7 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter an integer: ");
        int n = input.nextInt();
        switch (n % 2) {
            case 0:
                System.out.println("Even");
                break;
            case 1:
            case -1:
                System.out.println("Odd");
                break;
        }
    }
}
```

Output
Enter an integer: 12
Even

Exercise 08:

Write a program that reads an arithmetic operation op (+, -, * or /) and two numbers x and y, and performs the operation x op y. Two solutions: if and switch.

```
import java.util.Scanner;
public class Exercise8 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter an operation: ");
        char op = input.next().charAt(0);
        System.out.print("Enter x: ");
        double x = input.nextDouble();
        System.out.print("Enter y: ");
        double y = input.nextDouble();
        switch (op) {
            case '+':
                System.out.println(x + " + " + y + " = " + (x + y));
                break;
            case '-':
                System.out.println(x + " - " + y + " = " + (x - y));
                break;
            case '*':
                System.out.println(x + " * " + y + " = " + (x * y));
                break;
            case '/':
                System.out.println(x + " / " + y + " = " + (x / y));
                break;
            default:
                System.out.println("Invalid operation");
        }
    }
}
```

Output
Enter an operation: *
Enter x: 2
Enter y: 5
2.0 * 5.0 = 10.0

```
import java.util.Scanner;
public class Exercise8If {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter an operation: ");
        char op = input.next().charAt(0);
        System.out.print("Enter x: ");
        double x = input.nextDouble();
        System.out.print("Enter y: ");
        double y = input.nextDouble();
        if (op == '+')
            System.out.println(x + " + " + y + " = " + (x + y));
        else if (op == '-')
            System.out.println(x + " - " + y + " = " + (x - y));
        else if (op == '*')
            System.out.println(x + " * " + y + " = " + (x * y));
        else if (op == '/')
            System.out.println(x + " / " + y + " = " + (x / y));
        else
            System.out.println("Invalid operation");
    }
}
```

Output
Enter an operation: -
Enter x: 20
Enter y: 45.5
20.0 - 45.5 = -25.5

Exercise 09:

Write a method that computes the sum of the digits in an integer. Use the following method header:

public static int sumDigits(long n)

For example, sumDigits(234) returns 9

(Hint: Use the % operator to extract digits, and the / operator to remove the extracted digit). For instance, to extract 4 from 234, use 234 % 10 To remove 4 from 234, use 234 / 10.

Use a loop to repeatedly extract and remove the digit until all the digits are extracted.

Write also a test program that prompts the user to enter an integer and displays the sum of all its digits.

```

import java.util.Scanner;
public class Exercise9 {
    public static int sumDigits(long n) {
        int sum = 0;
        while (n != 0) {
            sum += n % 10;
            n /= 10;
        }
        return sum;
    }
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter an integer: ");
        long number = input.nextLong();
        System.out.println("Sum of digits = " + sumDigits(number));
    }
}

```

Output
Enter an integer: 234
Sum of digits = 9

Exercise 10:

A palindromic prime is a prime number and palindromic . Write a program that displays the first 100 palindromic prime numbers. Display 10 numbers per line, separated by exactly one space

```

public class Exercise10 {
    public static boolean isPrime(int n) {
        if (n < 2)
            return false;
        for (int i = 2; i <= Math.sqrt(n); i++) {
            if (n % i == 0)
                return false;
        }
        return true;
    }
    public static boolean isPalindrome(int n) {
        int original = n;
        int reverse = 0;
        while (n != 0) {
            reverse = reverse * 10 + n % 10;
            n /= 10;
        }
        return original == reverse;
    }
    public static void main(String[] args) {
        int count = 0;
        int number = 2;
        while (count < 100) {
            if (isPrime(number) && isPalindrome(number)) {
                System.out.print(number + " ");
                count++;
                if (count % 10 == 0)
                    System.out.println();
            }
            number++;
        }
    }
}

```

```

    }
}
}

```

Output (First Lines)

2 3 5 7 11 101 131 151 181 191 313 353 373 383 727 757 787 797 919 929

Exercise 11:

- Create an array of integers named List. Use the shorthand notation to fill it with ten values.
- Write the code to sum up all the element in the array.
- Write the code to find the highest value in the array.

```

public class Exercise11 {
    public static void main(String[] args) {
        int[] list = {12, 5, 18, 9, 25, 7, 30, 14, 3, 20};
        int sum = 0;
        for (int i = 0; i < list.length; i++) {
            sum += list[i];
        }
        System.out.println("Sum = " + sum);
        int max = list[0];
        for (int i = 1; i < list.length; i++) {
            if (list[i] > max)
                max = list[i];
        }
        System.out.println("Highest value = " + max);
    }
}

```

Output

Sum = 143 Highest value = 30

Exercise 12:

- (Find the average of the elements) Write a method that returns the average of the elements in an array of numbers. Use the following header:
public static double average(double[] array)
- (Count of elements below a key) Write a method that returns the number of elements in an array that is below a specific value. Use the following header:
public static int countAbove(double[] array, double key)
- (Find the range between max and min) Write a method that returns the difference between the highest and the lowest values in the array. Use the following header:
public static double findRange (double[] array)
- Write a test program that prompts the user to enter ten numbers, invokes the above methods to display the average of the numbers, the number of values above average, the number of values below average, and the range between the maximum and the minimum numbers.

```

import java.util.Scanner;
public class Exercise12 {
    public static double average(double[] array) {
        double sum = 0;
        for (int i = 0; i < array.length; i++)
            sum += array[i];
        return sum / array.length;
    }
    public static int countAbove(double[] array, double key) {
        int count = 0;
        for (int i = 0; i < array.length; i++) {
            if (array[i] > key)
                count++;
        }
        return count;
    }
    public static int countBelow(double[] array, double key) {
        int count = 0;
        for (int i = 0; i < array.length; i++) {
            if (array[i] < key)
                count++;
        }
        return count;
    }
    public static double findRange(double[] array) {
        double max = array[0];
        double min = array[0];
        for (int i = 1; i < array.length; i++) {
            if (array[i] > max)
                max = array[i];
            if (array[i] < min)
                min = array[i];
        }
        return max - min;
    }
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        double[] numbers = new double[10];
        System.out.println("Enter 10 numbers:");
        for (int i = 0; i < numbers.length; i++)
            numbers[i] = input.nextDouble();
        double avg = average(numbers);
        System.out.println("Average = " + avg);
        System.out.println("Above Average = " + countAbove(numbers, avg));
        System.out.println("Below Average = " + countBelow(numbers, avg));
        System.out.println("Range = " + findRange(numbers));
    }
}

```


Output
Enter 10 numbers:
5
7
3
10
8
4
2
6
9
1
Average = 5.5
Above Average = 5
Below Average = 5
Range = 9.0

Exercise 13:

Design a class named Triangle to represent a triangle. The class contains:

- Three double data fields named side1, side2, and side3 that specify the three sides of the triangle.
- A no-arg constructor that creates a default triangle. Use the value 1 for each of three sides.
- A constructor that creates a triangle with the specified side1, side2, and side3.
- A method named area() that returns the area of this triangle.
- A method named perimeter() that returns the perimeter of this triangle.

Note that the perimeter is the sum of the three sides and the area is calculated using the below formula:

$$s = (\text{side1} + \text{side2} + \text{side3}) / 2$$

$$\text{Area} = \text{radical}(s(s-s1)(s-s2)(s-s3))$$

1. Draw the UML diagram for the class Triangle.
2. Implement (Write the code for) the class Triangle.
3. Write an application (also known as a test program) that creates two Triangle objects - one using the no-arg constructor and the other with side1 4, side2 5, and side3 6. Display the area and perimeter of each triangle.

Triangle
- side1 : double - side2 : double - side3 : double
+ Triangle() + Triangle(double,double,double) + area() : double + perimeter() : double

```
public class Triangle {
    double side1;
    double side2;
    double side3;
    public Triangle() {
        side1 = side2 = side3 = 1.0;
    }
    public Triangle(double s1, double s2, double s3) {
        side1 = s1;
        side2 = s2;
        side3 = s3;
    }
}
```

```

public double perimeter() {
    return side1 + side2 + side3;
}
public double area() {
    double s = perimeter() / 2;
    return Math.sqrt(s * (s - side1) * (s - side2) * (s - side3));
}
}

```

```

public class TestTriangle {
    public static void main(String[] args) {
        Triangle t1 = new Triangle();
        Triangle t2 = new Triangle(4, 5, 6);
        System.out.println("Triangle 1");
        System.out.println("Area = " + t1.area());
        System.out.println("Perimeter = " + t1.perimeter());
        System.out.println();
        System.out.println("Triangle 2");
        System.out.println("Area = " + t2.area());
        System.out.println("Perimeter = " + t2.perimeter());
    }
}

```

Output
Triangle 1
Area = 0.4330127018922193
Perimeter = 3.0
Triangle 2
Area = 9.921567416492215
Perimeter = 15.0

Exercise 14:

(The Account class) Design a class named Account that contains:

- A private int data field named id for the account (default 0).
- A private double data field named balance for the account (default 0).
- A private double data field named annualInterestRate that stores the current interest rate (default 0). Assume all accounts have the same interest rate.
- A private Date data field named dateCreated that stores the date when the account was created.
- A no-arg constructor that creates a default account.
- A constructor that creates an account with the specified id and initial balance.
- The accessor and mutator methods for id, balance, and annualInterestRate.
- The accessor method for dateCreated.
- A method named getMonthlyInterestRate() that returns the monthly interest rate.
- A method named getMonthlyInterest() that returns the monthly interest.
- A method named withdraw that withdraws a specified amount from the account.
- A method named deposit that deposits a specified amount to the account.

Draw the UML diagram for the class and then implement the class. (Hint: The method getMonthlyInterest() is to return monthly interest, not the interest rate. Monthly interest is $\text{balance} * \text{monthlyInterestRate}$. $\text{monthlyInterestRate}$ is $\text{annualInterestRate} / 12$. Note that annualInterestRate is a percentage, e.g., like 4.5%. You need to divide it by 100.)

Write a test program that creates an Account object with an account ID of 1122, a balance of \$20,000, and an annual interest rate of 4.5%. Use the withdraw method to withdraw \$2,500, use the deposit method to deposit \$3,000, and print the balance, the monthly interest, and the date when this account was created.

Account
- id : int - balance : double - annualInterestRate : double - dateCreated : Date
+ Account() + Account(id:int, balance:double) + getId() : int + setId(id:int) : void + getBalance() : double + setBalance(balance:double) : void + getAnnualInterestRate() : double + setAnnualInterestRate(rate:double) : void + getDateCreated() : Date + getMonthlyInterestRate() : double + getMonthlyInterest() : double + withdraw(amount:double) : void + deposit(amount:double) : void

```
import java.util.Date;
public class Account {
    private int id = 0;
    private double balance = 0;
    private double annualInterestRate = 0;
    private Date dateCreated;
    public Account() {
        dateCreated = new Date();
    }
    public Account(int id, double balance) {
        this.id = id;
        this.balance = balance;
        dateCreated = new Date();
    }
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public double getBalance() {
        return balance;
    }
    public void setBalance(double balance) {
        this.balance = balance;
    }
    public double getAnnualInterestRate() {
        return annualInterestRate;
    }
    public void setAnnualInterestRate(double rate) {
        annualInterestRate = rate;
    }
}
```

```

    public Date getDateCreated() {
        return dateCreated;
    }
    public double getMonthlyInterestRate() {
        return annualInterestRate / 100 / 12;
    }
    public double getMonthlyInterest() {
        return balance * getMonthlyInterestRate();
    }
    public void withdraw(double amount) {
        balance -= amount;
    }
    public void deposit(double amount) {
        balance += amount;
    }
}

```

```

public class TestAccount {
    public static void main(String[] args) {
        Account account = new Account(1122, 20000);
        account.setAnnualInterestRate(4.5);
        account.withdraw(2500);
        account.deposit(3000);
        System.out.println("Balance: $" + account.getBalance());
        System.out.println("Monthly Interest: $"
            + account.getMonthlyInterest());
        System.out.println("Date Created: "
            + account.getDateCreated());
    }
}

```

Output

```

Balance: $20500.0
Monthly Interest: $76.875
Date Created: Fri Jul 24 09:00:00 GMT 2026

```

Exercise 15:

You can check whether a string is a substring of another string by using the `indexOf` method in the `String` class. Write your own method for this function. Write a program that prompts the user to enter two strings, and check whether the first string is a substring of the second

```

import java.util.Scanner;
public class Exercise15 {
    public static boolean isSubstring(String s1, String s2) {
        return s2.indexOf(s1) != -1;
    }
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter first string: ");
        String str1 = input.nextLine();
        System.out.print("Enter second string: ");
        String str2 = input.nextLine();
        if (isSubstring(str1, str2))
            System.out.println("The first string is a substring of the second");
        else
            System.out.println("The first string is not a substring of the second");
    }
}

```

```
}
```

Output 1

```
Enter first string: Java
Enter second string: I love Java programming
The first string is a substring of the second
```

Output 2

```
Enter first string: Python
Enter second string: I love Java programming
The first string is not a substring of the second
```

Exercise 16: (Inheritance)

Create a class named Person that contains:

A private String data field named name.

A private int data field named age.

A constructor that initializes name and age.

A method displayInfo() that displays the person's information.

Create a subclass named Student that extends Person and contains:

A private String data field named major.

A constructor that initializes name, age, and major using super.

Override the method displayInfo() to display student information.

Write a test program that creates a Person object and a Student object and displays their information.

```
public class Person {
    private String name;
    private int age;
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
    public void displayInfo() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}
```

```
public class Student extends Person {
    private String major;
    public Student(String name, int age, String major) {
        super(name, age);
        this.major = major;
    }
    @Override
    public void displayInfo() {
        super.displayInfo();
        System.out.println("Major: " + major);
    }
}
```

```
public class TestInheritance {
    public static void main(String[] args) {
        Person p1 = new Person("Ali", 25);
        Student s1 = new Student("Ahmad", 21, "Software Engineering");
        System.out.println("Person Information:");
    }
}
```

```

        p1.displayInfo();
        System.out.println();
        System.out.println("Student Information:");
        s1.displayInfo();
    }
}

```

Output

Person Information:

Name: Ali

Age: 25

Student Information:

Name: Ahmad

Age: 21

Major: Software Engineering

Exercise 17:

Example Illustrating Polymorphism, Casting, and the instanceof Operator:

```

public class Application {
    public static void main(String[] args) {
        GeometricObject[] g = new GeometricObject[10];
        for(int i = 0 ; i < g.length ; i += 2){
            g[i] = new GeometricObject();
            g[i+1] = new Shape(3 + i, "Red", false);
        }
        Shape[] s;
        s = getShapes(g);
        for (int i = 0 ; i < s.length ; i++)
            System.out.println(s[i]);
        } // end main
    // Continued in next slide
    public static Shape[] getShapes(GeometricObject[] g){
        int numberOfShapeObjects=0;
        for(int i = 0 ; i < g.length ; i++)
            if(g[i] instanceof Shape) // instanceof
                numberOfShapeObjects++;
        Shape[] shapes = new Shape[numberOfShapeObjects];
        for(int i = 0 , k = 0; i < g.length ; i++)
            if(g[i] instanceof Shape){ // instanceof
                shapes[k] = (Shape)g[i]; // casting
                k++;
            }
        return shapes;
    } // end getShapes
} // end Application

```

Exercise 18: ArrayList

Write a program that creates an ArrayList of integers. The program should:

Add five numbers to the list.

Display all elements.

Remove an element from the list.

Search for a specific number.

```
import java.util.ArrayList;
import java.util.Scanner;
public class Exercise18 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        ArrayList<Integer> numbers = new ArrayList<>();
        // Add elements
        numbers.add(10);
        numbers.add(20);
        numbers.add(30);
        numbers.add(40);
        numbers.add(50);
        System.out.println("ArrayList: " + numbers);
        // Remove element
        numbers.remove(2);
        System.out.println("After removing index 2: " + numbers);
        // Search element
        System.out.print("Enter a number to search: ");
        int n = input.nextInt();
        if (numbers.contains(n))
            System.out.println("Number exists");
        else
            System.out.println("Number does not exist");
    }
}
```

Output

```
ArrayList: [10, 20, 30, 40, 50]
After removing index 2: [10, 20, 40, 50]
Enter a number to search: 40
Number exists
```

Exercise 19: Exception Handling

Write a program that asks the user to enter two numbers and divides them. Handle possible exceptions using try-catch.

```
import java.util.Scanner;
public class Exercise19 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        try {
            System.out.print("Enter first number: ");
            int n1 = input.nextInt();
            System.out.print("Enter second number: ");
            int n2 = input.nextInt();
            int result = n1 / n2;
            System.out.println("Result = " + result);
        }
        catch (ArithmeticException e) {
            System.out.println("Error: Cannot divide by zero");
        }
    }
}
```

```

        catch (Exception e) {
            System.out.println("Invalid input");
        }
        finally {
            System.out.println("Program finished");
        }
    }
}

```

Output 1

```

Enter first number: 20
Enter second number: 5
Result = 4
Program finished

```

Output 2

```

Enter first number: 20
Enter second number: 0
Error: Cannot divide by zero
Program finished

```

Exercise 20: Text I/O and File Handling

Write a program that creates a text file named students.txt, writes student names into the file, then reads and displays the file contents.

```

import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;
public class Exercise20 {
    public static void main(String[] args) {
        // Writing to file
        try {
            FileWriter writer = new FileWriter("students.txt");
            writer.write("Ali\n");
            writer.write("Ahmad\n");
            writer.write("Sara\n");
            writer.close();
            System.out.println("File created successfully");
        }
        catch (IOException e) {
            System.out.println("Error writing file");
        }
        // Reading from file
        try {
            File file = new File("students.txt");
            Scanner reader = new Scanner(file);
            System.out.println("Students:");
            while (reader.hasNextLine()) {
                String name = reader.nextLine();
                System.out.println(name);
            }
            reader.close();
        }
        catch (IOException e) {
            System.out.println("File not found");
        }
    }
}

```



```
}  
}
```

Output

```
File created successfully  
Students:  
Ali  
Ahmad  
Sara
```

Exercise 21: Enum in Java

Create an enum named Day that contains the days of the week.

Write a program that asks the user to enter a day and displays whether it is a weekday or weekend using enum.

```
import java.util.Scanner;  
public class Exercise21 {  
    enum Day {  
        MONDAY,  
        TUESDAY,  
        WEDNESDAY,  
        THURSDAY,  
        FRIDAY,  
        SATURDAY,  
        SUNDAY  
    }  
    public static void main(String[] args) {  
        Scanner input = new Scanner(System.in);  
        System.out.print("Enter a day: ");  
        String dayInput = input.next().toUpperCase();  
        Day day = Day.valueOf(dayInput);  
        switch(day) {  
            case SATURDAY:  
            case SUNDAY:  
                System.out.println("Weekend");  
                break;  
            default:  
                System.out.println("Weekday");  
        }  
    }  
}
```

Output 1

```
Enter a day: Monday  
Weekday
```

Output 2

```
Enter a day: Sunday  
Weekend
```

Exercise 22:

Two words are anagrams if one word can be formed by rearranging all the letters of the other word, for example: treason, senator. A word is represented as a linked list with one letter per node of the list. Write a method that, given two linked lists w1 and w2, each representing a word of lowercase letters, returns true if the words are anagrams and false if they are not.

Base your algorithm on the following: for each letter in w1, search w2 for it; if found, delete it and continue; otherwise, return false.

```
import java.util.LinkedList;
public class Exercise22 {
    public static boolean isAnagram(LinkedList<Character> w1,
                                    LinkedList<Character> w2) {
        if (w1.size() != w2.size())
            return false;
        LinkedList<Character> temp = new LinkedList<>(w2);
        for (char c : w1) {
            if (temp.contains(c)) {
                temp.remove((Character)c);
            }
            else {
                return false;
            }
        }
        return true;
    }
    public static void main(String[] args) {
        LinkedList<Character> word1 = new LinkedList<>();
        word1.add('t');
        word1.add('r');
        word1.add('e');
        word1.add('a');
        word1.add('s');
        word1.add('o');
        word1.add('n');
        LinkedList<Character> word2 = new LinkedList<>();
        word2.add('s');
        word2.add('e');
        word2.add('n');
        word2.add('a');
        word2.add('t');
        word2.add('o');
        word2.add('r');
        System.out.println(isAnagram(word1, word2));
    }
}
```

Output

true

Exercise 23:

Add a method in the class LinkedList named sumValues() that returns the sum of the values in all nodes of a linked list. Create an application in which you:

1. Create a linked list and add to it the values 6, 4, 10, 9, 1.
2. Call the method sumValues to output the sum of the values in the linked list (the sum will be 30).

```
import java.util.LinkedList;
public class MyLinkedList {
    LinkedList<Integer> list = new LinkedList<>();
    public int sumValues() {
        int sum = 0;
        for (int value : list) {
            sum += value;
        }
        return sum;
    }
    public static void main(String[] args) {
        MyLinkedList obj = new MyLinkedList();
        obj.list.add(6);
        obj.list.add(4);
        obj.list.add(10);
        obj.list.add(9);
        obj.list.add(1);
        System.out.println("Sum = " + obj.sumValues());
    }
}
```

Output

Sum = 30

Exercise 24:

Add a method in the application named sumNumbers() that receives as a parameter from the main method a linked list and returns the sum of the values in all nodes.

Create an application in which you:

1. Create a linked list and add to it the values 6, 4, 10, 9, 1.
2. Call the method sumNumbers to output the sum of the values in the linked list (the sum will be 30).

```
import java.util.LinkedList;
public class Exercise24 {
    public static int sumNumbers(LinkedList<Integer> list) {
        int sum = 0;
        for (int number : list) {
            sum += number;
        }
        return sum;
    }
    public static void main(String[] args) {
        LinkedList<Integer> numbers = new LinkedList<>();
        numbers.add(6);
        numbers.add(4);
        numbers.add(10);
        numbers.add(9);
        numbers.add(1);
        System.out.println("Sum = " + sumNumbers(numbers));
    }
}
```

Output
Sum = 30

Exercise 25:

A frequently occurring problem that is detected by compilers is to determine if the brackets of an expression are properly balanced and included one in the other. For example, the expression $((a+b)*c)+(u*v)$ is well balanced and properly written while the expressions $(a+b)*c$ and $(a+b)*c)+b($ are not. Write an application in which you:

- Write a boolean method named `check` that receives as a parameter a `String` expression and checks if the expression is well balanced and properly written or not. The header of the method is: **public static boolean check(String expression)**
- Write a main method to test the above method.

Note: Use the stack ADT operations to solve this problem. You are not allowed to use arrays or counters.

```
import java.util.Stack;
import java.util.Scanner;
public class Exercise25 {
    public static boolean check(String expression) {
        Stack<Character> stack = new Stack<>();
        for (int i = 0; i < expression.length(); i++) {
            char ch = expression.charAt(i);
            // Add opening brackets
            if (ch == '(') {
                stack.push(ch);
            }
            // Check closing bracket
            else if (ch == ')') {
                if (stack.empty()) {
                    return false;
                }
                stack.pop();
            }
        }
        return stack.empty();
    }
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter expression: ");
        String exp = input.nextLine();
        if (check(exp))
            System.out.println("Balanced expression");
        else
            System.out.println("Not balanced expression");
    }
}
```

Output 1
Enter expression: $((a+b)*c)+(u*v)$
Balanced expression

Output 2
Enter expression: $(a+b)*c$
Not balanced expression

Exercise 26:

In a bank, customers wait in a queue (q0) for their turn to complete a transaction. Customers who come first are served first. At a certain point when there are lots of customers waiting, a decision is taken to split the queue elements into two empty queues q1 and q2. The customers are split based on the alphabetical order of their names. Customers with first name starting A-H are added to q1 while the others (first name starting I- Z) are added to q2.

In an application, write a method named splitQueue (in an application) that accepts as parameters 3 queues, the original queue (q0) and two empty queues (q1 and q2) to be filled as described above. Write also a main method in which you read the names of customers from the user, fill them into q0 and call the method splitQueue to split q0 elements into q1 and q2. After the split display the elements of q1 first then those of q2. The header of the method is:

public static void splitQueue (Queue q0, Queue q1, Queue q2)

Note: Make sure you do the necessary modifications to allow the queue to store Strings (first names).

```
import java.util.LinkedList;
import java.util.Queue;
import java.util.Scanner;
public class Exercise26 {
    public static void splitQueue(Queue<String> q0, Queue<String> q1, Queue<String> q2) {
        while (!q0.isEmpty()) {
            String name = q0.remove();
            char firstLetter = name.charAt(0);
            if (firstLetter >= 'A' && firstLetter <= 'H') {
                q1.add(name);
            }
            else {
                q2.add(name);
            }
        }
    }
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        Queue<String> q0 = new LinkedList<>();
        Queue<String> q1 = new LinkedList<>();
        Queue<String> q2 = new LinkedList<>();
        System.out.print("Enter number of customers: ");
        int n = input.nextInt();
        input.nextLine();
        for (int i = 0; i < n; i++) {
            System.out.print("Enter customer name: ");
            String name = input.nextLine();
            q0.add(name);
        }
        splitQueue(q0, q1, q2);
        System.out.println("\nQueue 1 (A-H):");
        while (!q1.isEmpty()) {
            System.out.println(q1.remove());
        }
        System.out.println("\nQueue 2 (I-Z):");
        while (!q2.isEmpty()) {
            System.out.println(q2.remove());
        }
    }
}
```

```
}
```

Output

Enter number of customers: 6

Enter customer name: Ali

Enter customer name: Sara

Enter customer name: Hassan

Enter customer name: Omar

Enter customer name: Ahmad

Enter customer name: Lina

Queue 1 (A-H):

Ali

Hassan

Ahmad

Exercise 27: (Big O)

Prove that $n^3 + 1000 = O(n^3)$

According to the definition of Big-O:

$f(n) \leq c \times g(n)$ where $f(n) = n^3 + 1000$ $g(n) = n^3$

We need to find constants c and n_0 .

Choose:

$c = 2$ and: $n_0 = 10$

For every $n \geq 10$ we have:

$n^3 \geq 1000$

Therefore $n^3 + 1000 \leq n^3 + n^3$

So $n^3 + 1000 \leq 2n^3$

This means $f(n) \leq c \times g(n)$

Therefore $n^3 + 1000 = O(n^3)$

Final Answer:

$c = 2$

$n_0 = 10$

Exercise 28:

Determine the running time of the following code in terms of Big-Oh notation. Show the steps in details.

```
public static void method(int n) {  
    int[] a = new int[n];  
    for (int i = 0; i < n; i++)  
        a[i] = in.nextInt();  
    for (int i = 0; i < n; i++)  
        System.out.println(a[i]);  
}
```

Step 1: Creating the array `int[] a = new int[n];` $O(1)$

Step 2: First loop `for (int i = 0; i < n; i++)` $\rightarrow O(n)$

Step 3: Second loop `for (int i = 0; i < n; i++)` $\rightarrow O(n)$

The operation inside the loop takes constant time:

Step 4: Total Running Time $O(1) + O(n) + O(n) \rightarrow O(n^2)$

EXERCISE 29: Recursive oddSum Method

Write a recursive method named oddSum that takes a positive odd integer n and returns the sum of odd integers from 1 to n.

Example , oddSum(5) = 1 + 3 + 5 = 9

```
import java.util.Scanner;
public class Exercise29 {
    public static int oddSum(int n) {
        if (n == 1)
            return 1;
        return n + oddSum(n - 2);
    }
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter an odd number: ");
        int n = input.nextInt();
        System.out.println("Sum = " + oddSum(n));
    }
}
```

Output:

Enter an odd number: 7

Sum = 16

Exercise 30:

Write a recursive method named oddSum that takes a positive odd integer n and returns the sum of odd integers from 1 to n. Start with a base case, and use temporary variables to debug your solution. You might find it helpful to print the value of n each time oddSum is invoked. Write a main method to test the above method.

Write a recursive method named oddSum that takes a positive odd integer n and returns the sum of odd integers from 1 to n.

Example: oddSum(7) = 1 + 3 + 5 + 7 = 16

```
import java.util.Scanner;
public class Exercise29 {
    public static int oddSum(int n) {
        if (n == 1)
            return 1;
        return n + oddSum(n - 2);
    }
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter a positive odd number: ");
        int n = input.nextInt();
        System.out.println("sum of odd numbers = " + oddSum(n));
    }
}
```

Output:

Enter a positive odd number: 9

Sum of odd numbers = 25

Exercise 31:

A palindrome is a word that reads the same both forward and backward, like “otto” and “palindromeemordnilap”. Here’s one way to test whether a string is a palindrome:

A single letter is a palindrome, a two-letter word is a palindrome if the letters are the same, and any other word is a palindrome if the first letter is the same as the last and the middle is a palindrome.

Write a recursive method named isPalindrome that takes a string as a parameter and returns true

if the word is a palindrome and false otherwise.

Write a main method to test the above method.

```
import java.util.Scanner;
public class Exercise30 {
    public static boolean isPalindrome(String word) {
        if (word.length() <= 1)
            return true;
        if (word.charAt(0) != word.charAt(word.length() - 1))
            return false;
        return isPalindrome(word.substring(1, word.length() - 1));
    }
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter a word: ");
        String word = input.nextLine();
        if (isPalindrome(word))
            System.out.println("The word is a palindrome");
        else
            System.out.println("The word is not a palindrome");
    }
}
```

Output 1:

Enter a word: otto

The word is a palindrome

Output 2:

Enter a word: hello

The word is not a palindrome

Exercise 32: (Tree&recursion)

Write a recursive method that takes a binary tree and checks if it is a full binary tree. A full binary tree is a tree in

which every node other than the leaves has exactly two children. In other words, a binary tree is full if each node

has 2 children or 0 children (leaf).

```
Node Class:
class TreeNode {
    int value;
    TreeNode left;
    TreeNode right;
    public TreeNode(int value) {
        this.value = value;
        left = null;
        right = null;
    }
}
```


Recursive Method:

```
public static boolean isFullBinaryTree(TreeNode root) {  
    if (root == null)  
        return true;  
    if (root.left == null && root.right == null)  
        return true;  
    if (root.left == null || root.right == null)  
        return false;  
    return isFullBinaryTree(root.left) && isFullBinaryTree(root.right);  
}
```

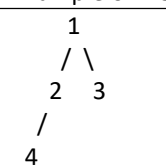
Test Program:

```
public class Exercise31 {  
    public static void main(String[] args) {  
        TreeNode root = new TreeNode(1);  
        root.left = new TreeNode(2);  
        root.right = new TreeNode(3);  
        root.left.left = new TreeNode(4);  
        root.left.right = new TreeNode(5);  
        root.right.left = new TreeNode(6);  
        root.right.right = new TreeNode(7);  
        if (isFullBinaryTree(root))  
            System.out.println("The tree is a full binary tree");  
        else  
            System.out.println("The tree is not a full binary tree");  
    }  
    public static boolean isFullBinaryTree(TreeNode root) {  
        if (root == null)  
            return true;  
        if (root.left == null && root.right == null)  
            return true;  
        if (root.left == null || root.right == null)  
            return false;  
        return isFullBinaryTree(root.left) &&  
            isFullBinaryTree(root.right);  
    }  
}
```

Output:

The tree is a full binary tree

Example of NOT Full Binary Tree:



Output:

The tree is not a full binary tree

EXERCISE 33: Graphs (Graph Traversal)

Create a graph using adjacency lists and implement Depth First Search (DFS) to visit all vertices.

```
import java.util.*;
public class Exercise32 {
    private int vertices;
    private LinkedList<Integer>[] adjacencyList;
    public Exercise32(int vertices) {
        this.vertices = vertices;
        adjacencyList = new LinkedList[vertices];
        for (int i = 0; i < vertices; i++)
            adjacencyList[i] = new LinkedList<>();
    }
    public void addEdge(int source, int destination) {
        adjacencyList[source].add(destination);
    }
    public void DFS(int start) {
        boolean[] visited = new boolean[vertices];
        DFSUtil(start, visited);
    }
    private void DFSUtil(int vertex, boolean[] visited) {
        visited[vertex] = true;
        System.out.print(vertex + " ");
        for (int neighbor : adjacencyList[vertex]) {
            if (!visited[neighbor])
                DFSUtil(neighbor, visited);
        }
    }
    public static void main(String[] args) {
        Exercise32 graph = new Exercise32(5);
        graph.addEdge(0,1);
        graph.addEdge(0,2);
        graph.addEdge(1,3);
        graph.addEdge(2,4);
        System.out.println("DFS Traversal:");
        graph.DFS(0);
    }
}
```

Output:

DFS Traversal: 0 1 3 2 4

EXERCISE 34: HashMap and HashTable

Create a program that stores student names and their grades using HashMap, then searches for a student grade.

```
import java.util.HashMap; import java.util.Scanner;
public class Exercise33 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        HashMap<String, Integer> students = new HashMap<>();
        students.put("Ali", 90);
        students.put("Ahmad", 85);
        students.put("Sara", 95);
        System.out.println("Students Grades:");
        System.out.println(students);
        System.out.print("Enter student name: ");
    }
}
```

```

String name = input.nextLine();
if (students.containsKey(name))
    System.out.println("Grade = " + students.get(name));
else
    System.out.println("Student not found");
}
}

```

Output:

```

Students Grades:
{Ali=90, Ahmad=85, Sara=95}
Enter student name:
Sara
Grade = 95

```

EXERCISE 35: Vector

Create a Vector that stores numbers. Add elements, remove an element, and display the vector.

```

import java.util.Vector;
public class Exercise34 {
    public static void main(String[] args) {
        Vector<Integer> numbers = new Vector<>();
        numbers.add(10);
        numbers.add(20);
        numbers.add(30);
        numbers.add(40);
        System.out.println("Vector: " + numbers);
        numbers.remove(2);
        System.out.println("After removing element: " + numbers);
        System.out.println("First element: " + numbers.firstElement());
    }
}

```

Output

```

Vector: [10, 20, 30, 40]

```